FACULDADE DE TECNOLOGIA DE SINOP

Compiladores

ENGENHARIA DE COMPUTAÇÃO
FASTECH SINOP

Prof. Ramilton Costa Gomes Júnior

FACULDADE DE TECNOLOGIA DE SINOP

ENGENHARIA DE COMPUTAÇÃO FASTECH SINOP

Agenda

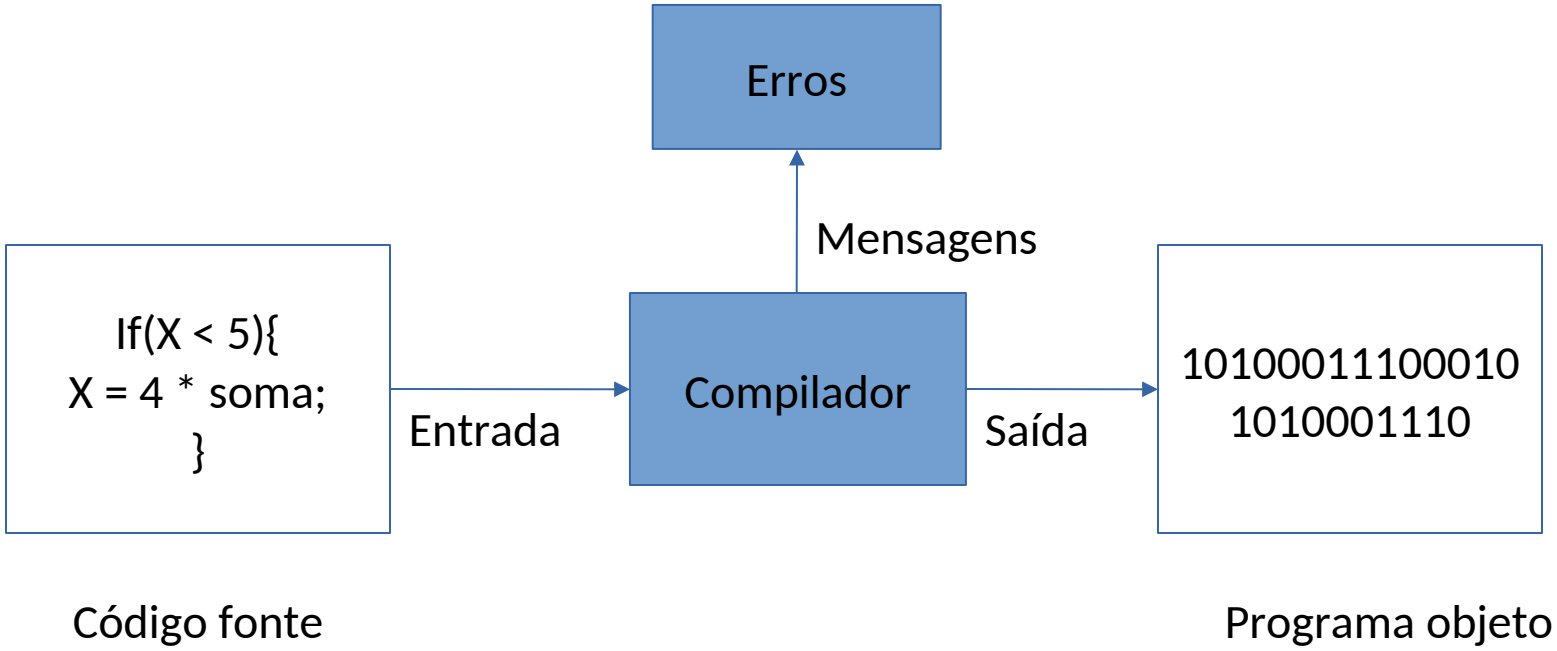
- Apresentação do plano de ensino;
- Introdução a compiladores;
- Visão geral do processo de compilação;
- Comparação entre compiladores e interpretadores.

Introdução a Compiladores

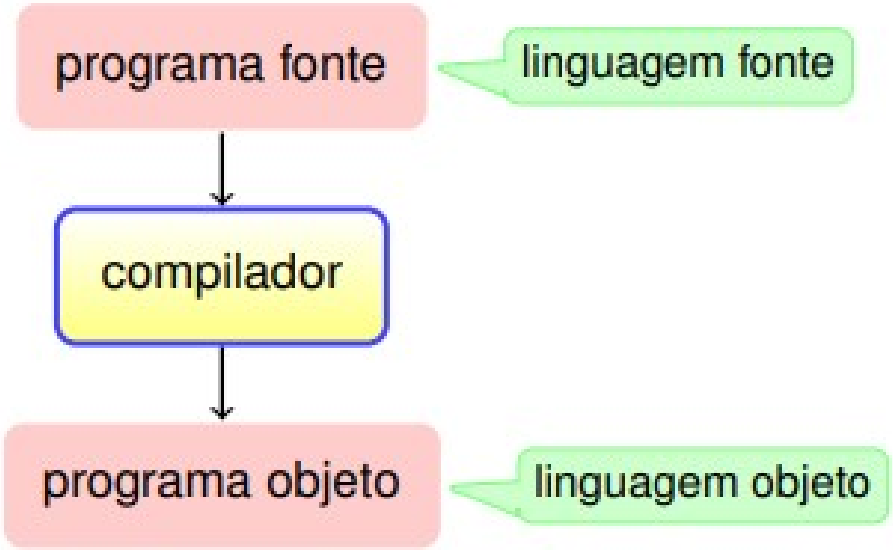
- O que são compiladores?

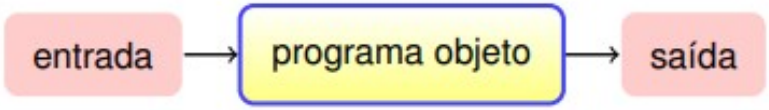
Compiladores são programas de computador que traduzem código fonte escrito em uma linguagem de programação de alto nível para uma linguagem de baixo nível.

- **O que são compiladores?**



- O que são compiladores?





- Processo de compilação

1. **Análise Léxica:** O compilador lê o código fonte e divide-o em unidades menores chamadas tokens, que representam elementos sintáticos, como palavras-chave, identificadores e operadores.

2. **Análise Sintática:** O compilador verifica a estrutura do código para garantir que ele siga as regras gramaticais da linguagem. Isso é feito por meio da construção de uma árvore de sintaxe abstrata (AST).

- Processo de compilação

3. Análise Semântica: Nessa fase, o compilador valida o significado do código, checando inconsistências e aplicando as regras de tipagem e outras restrições semânticas.

4. Otimização: O código intermediário pode ser otimizado para melhorar o desempenho, como eliminar instruções redundantes ou melhorar o uso de recursos.

- Processo de compilação

5. **Geração de Código:** O compilador converte o código otimizado em um formato que a máquina possa entender, gerando código de máquina ou linguagem de montagem.

6. **Linkagem:** Essa etapa combina o código gerado com bibliotecas e outros módulos necessários, criando um executável completo que pode ser executado no sistema operacional.

- Importância dos compiladores

- Facilitam a Programação;
- Portabilidade;
- Otimização de Código;
- Verificação de Erros;
- Segurança;
- Abstração;
- Desenvolvimento de Novas Linguagens;
- Suporte à Inovação.

- Visão geral do processo de compilação

- O processo de compilação de um programa de linguagem de alto nível em uma linguagem de máquina pode consumir uma quantidade considerável de tempo do computador;
- interpretadores executam linguagens de alto nível diretamente, embora sejam mais lentos que programas compiladores;

- Visão geral do processo de compilação

1)Análise Léxica:

- Entrada: Código fonte.
- Saída: Tokens.

2)Análise Sintática

- Entrada: Tokens.
- Saída: Árvore de Sintaxe Abstrata

- Visão geral do processo de compilação

3)Análise Semântica:

- Entrada: Árvore de Sintaxe Abstrata.
- Saída: Representação semântica ou erro de semântica.

4)Otimização

- Entrada: Representação intermediária.
- Saída: Código otimizado

- Visão geral do processo de compilação

Otimização

$$t_1 = \textbf{inttofloat(60)}$$
$$t_2 = id_3 * t_1$$
$$t_3 = id_2 + t_2$$
$$id_1 = t_3$$

$\longrightarrow$

$$t_1 = id_3 * \textbf{60.0}$$
$$id_1 = id_2 + t_1$$

- Visão geral do processo de compilação

5)Geração de código:

- Entrada: Código otimizado.
- Saída: Código de máquina ou linguagem de montagem.

6)Linkagem

- Entrada: Código de máquina e bibliotecas externas.
- Saída: Executável ou bibliotecas

- Visão geral do processo de compilação

Geração de código

$$t_1 = id_3 * 60.0$$
$$id_1 = id_2 + t_1$$

LDF R2, id_3
MULF R2, R2, #60.0
LDF R1, id_2
ADDF R1, R1, R2
STF id_1**, R1**

- Visão geral do processo de compilação

7)Carregamento:

- Entrada: Executável.
- Saída: Programa em execução.

- Histórico e evolução

- Antigamente
  - Programação em código de máquina;
- Primeiros compiladores começaram a surgir em 1950;
- Inicialmente, fizeram tradução de fórmulas aritméticas em código de máquina;

- Histórico e evolução

- 1º – Fortran
  - Permitia declaração de identificadores com no máximo 6 caracteres;
  - Formula Translation System;

- Histórico e evolução

- 1957: Fortran – primeiros compiladores de processar fórmulas;
- 1960: Algol – primeiro a usar gramática na forma de Backus, estrutras de blocos, recursão;
- 1970: Pascal - tipos definidos pelos usuários;
- 1985: C++ - orientação a objetos, exceções;
- 1995: Java – compilação instantânea (traduz bytecodes para código de máquinas e executa).

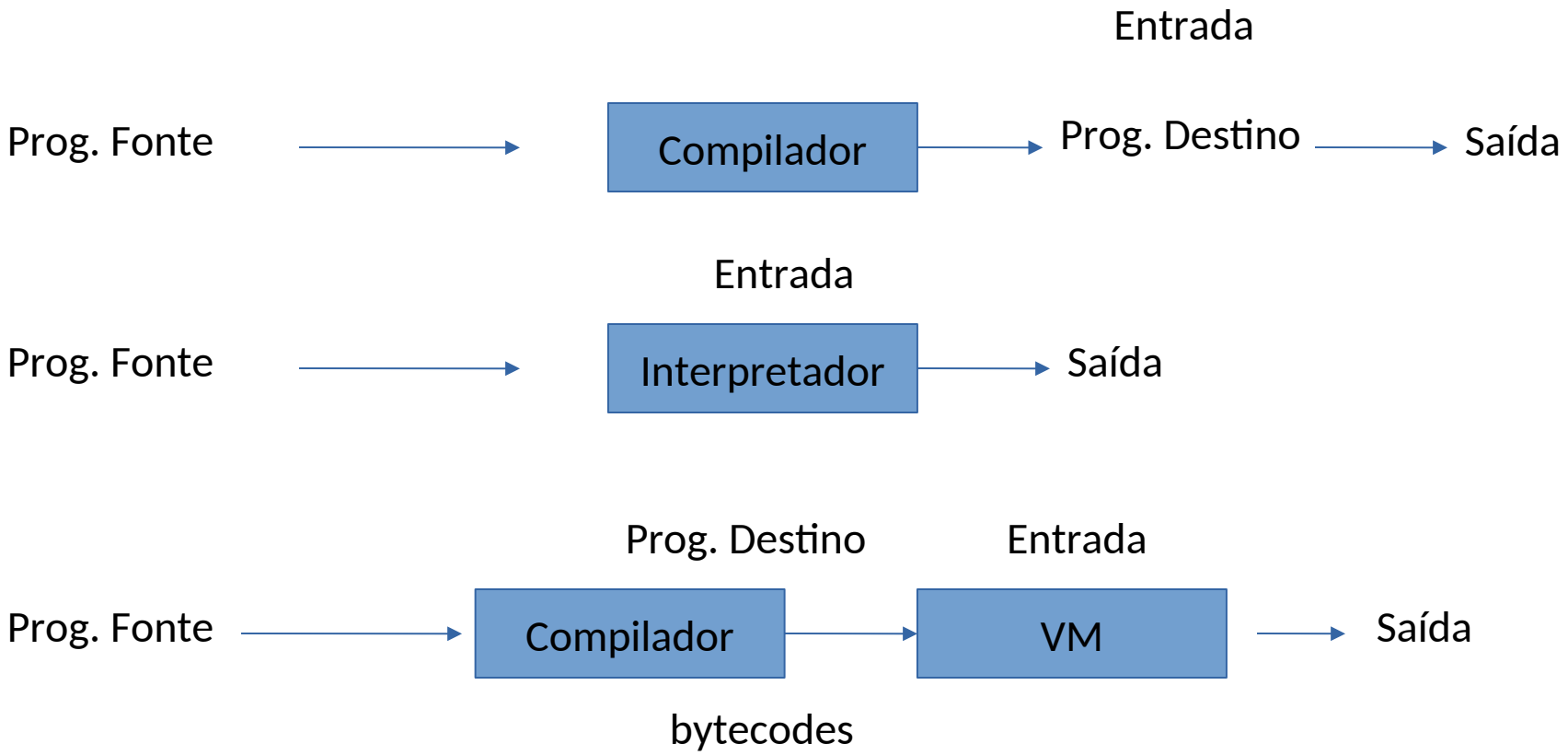
- Histórico e evolução

- Surge programas para produção automática de compiladores.
  - C:
    - Lex, Flex
      - Lex: Gerador de nalisadores léxicos (unix);
      - Flex: Geradores de analisadores léxicos (linux, win);
  - C++:

- Histórico e evolução

- C++:
  - Flex++, Flexx;
- Pascal
  - Tply – Tp lex/ Yacc;
- Java
  - Jflex: Gerador de analisadores léxicos
  - JCUP: Gerador de analisadores sintáticos

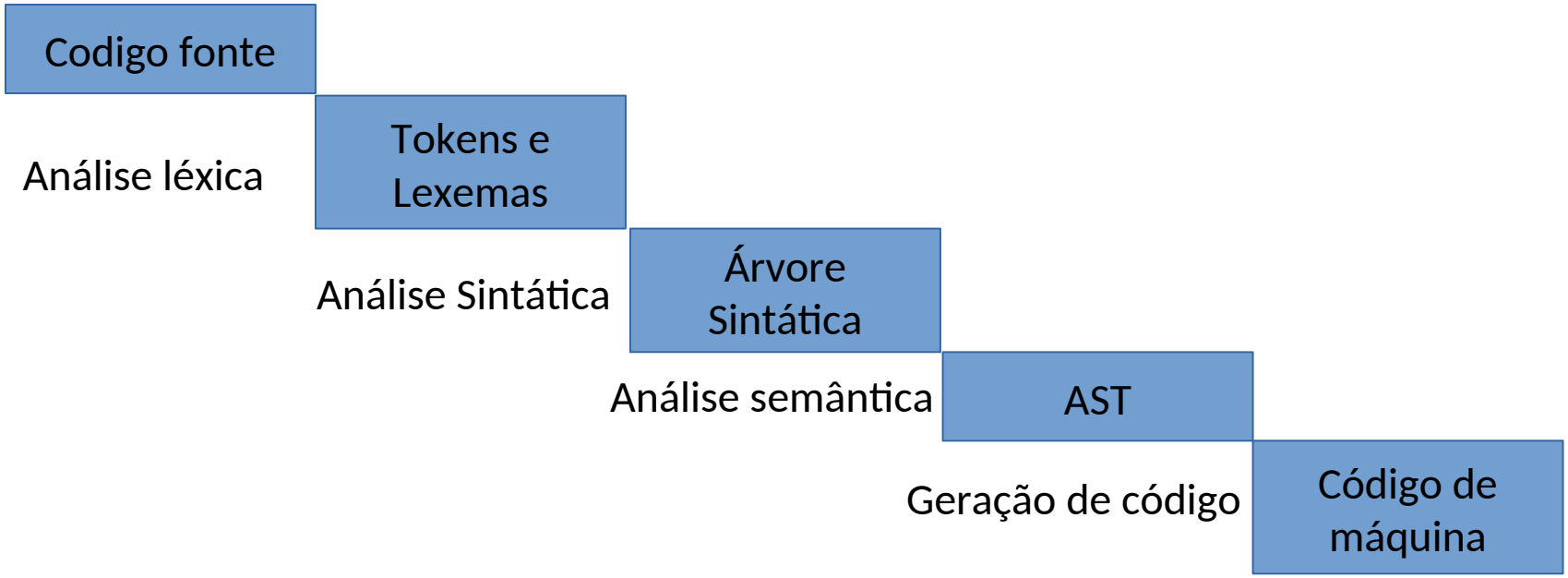
- **Compilador, interpretador e máquina virtual**



- Comparação entre compiladores e interpretadores

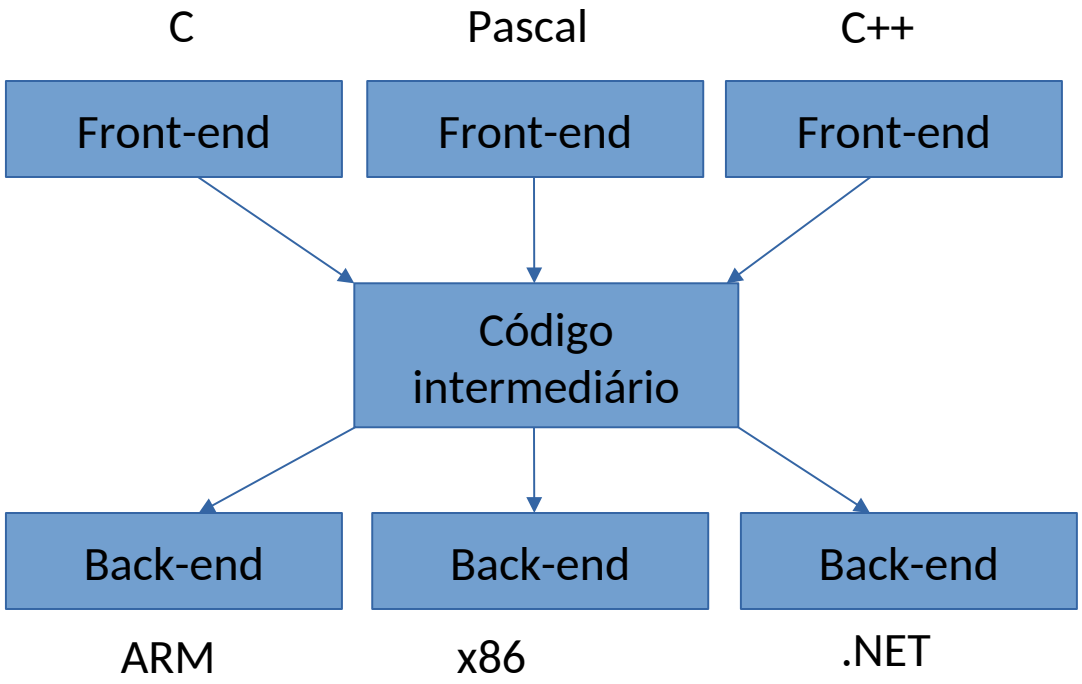
- Método de execução;
- Velocidade de Execução;
- Erros;
- Portabilidade;
- Uso de Recursos;
- Aplicações.

- **Visão geral do processo de compilação**



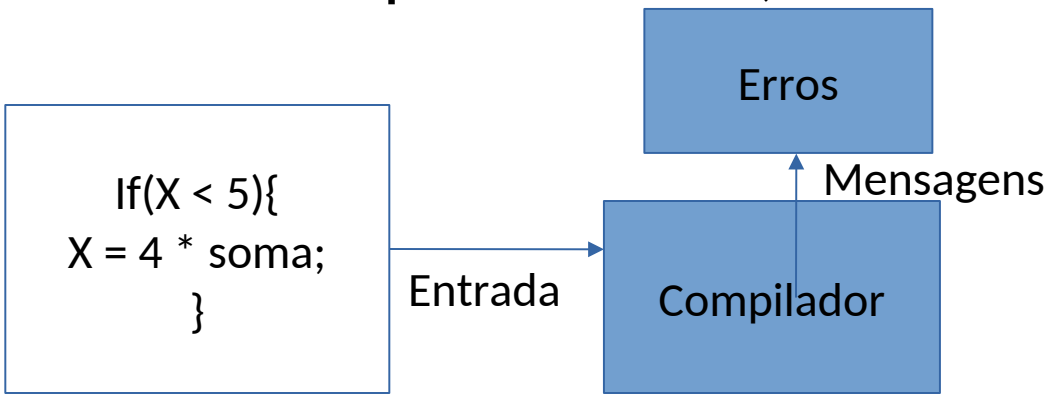
- Visão geral do processo de compilação

- Duas etapas
  - Análise (front-end);
  - Síntase (back-end);



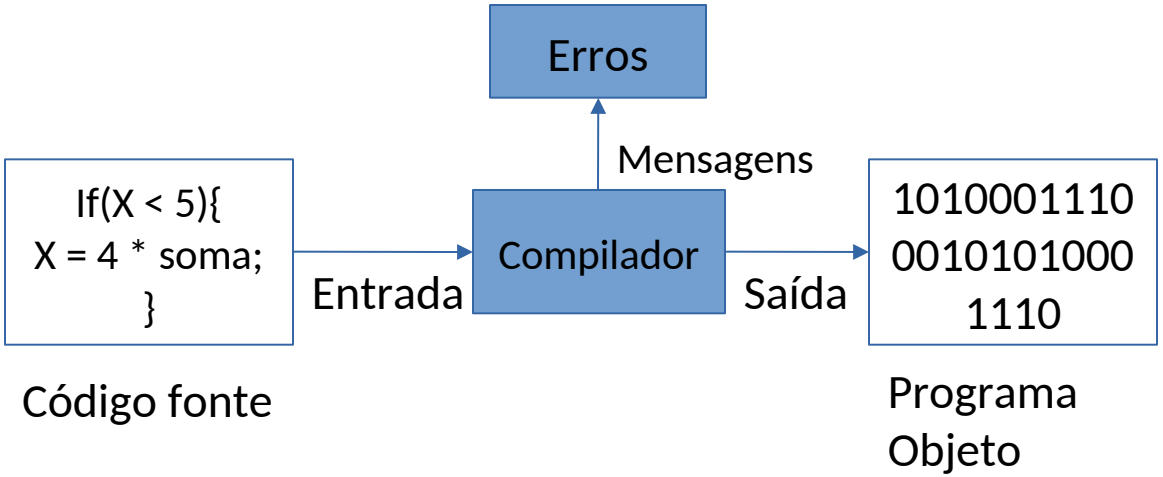
- Visão geral do processo de compilação

- Duas etapas
  - Análise (front-end)
    - Cria representações intermediárias do programa;
    - Verifica presença de certos tipos de erros;



- Visão geral do processo de compilação

- Duas etapas
  - Sintase (Back-end)
    - Cria programa destino a partir de representações intermediárias;



- **Comparação entre compiladores e interpretadores**
  - Por que C gera um arquivo e Ruby não?

- **Comparação entre compiladores e interpretadores**

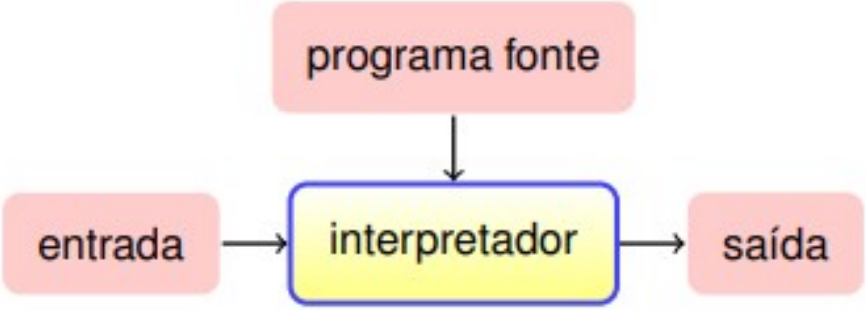
- Compiladores
  - Verifica a declaração
  - Gera o assembly
  - Executa o assembly
  - Erro de compilação
    - Não executa as declarações

- **Comparação entre compiladores e interpretadores**

- Compiladores
  - Vantagens
    - O código compilado é mais rápido de ser acessado;
    - Permite otimização do código por parte do compilador;
    - Compila o código somente se estiver sem algum erro.

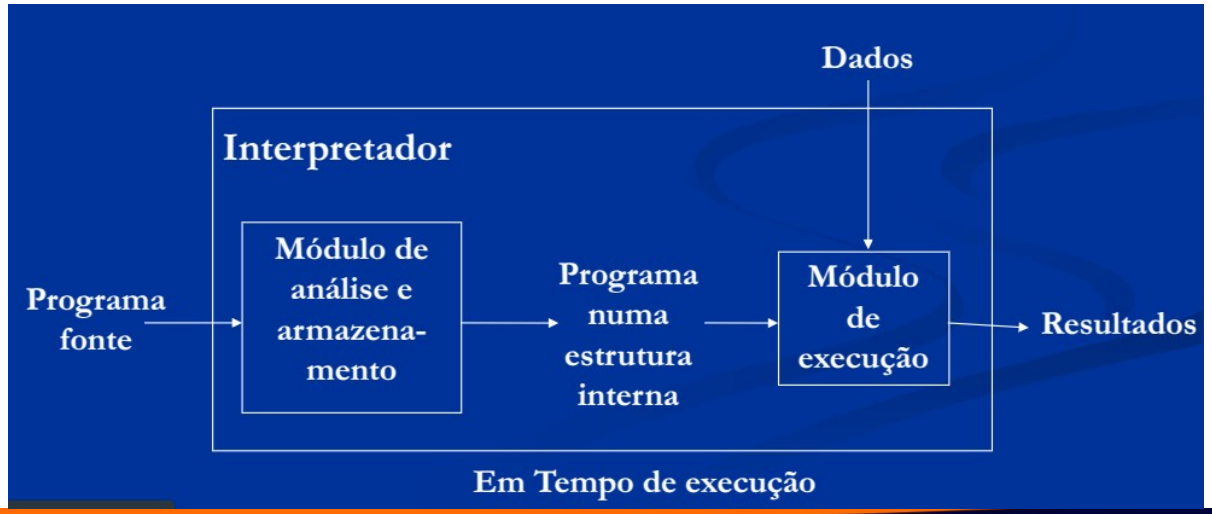
- **Comparação entre compiladores e interpretadores**

- Interpretador
  - Verifica a declaração
  - Executa a declaração
  - Erro de interpretação
    - Executa as declarações

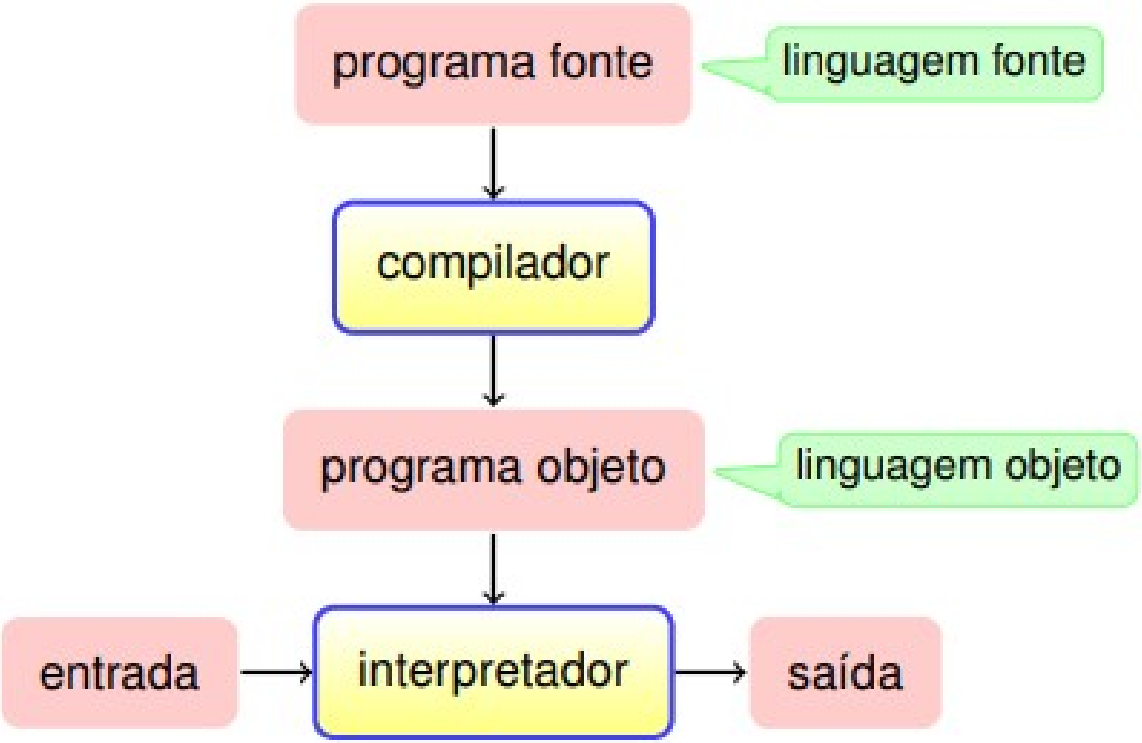


- **Comparação entre compiladores e interpretadores**

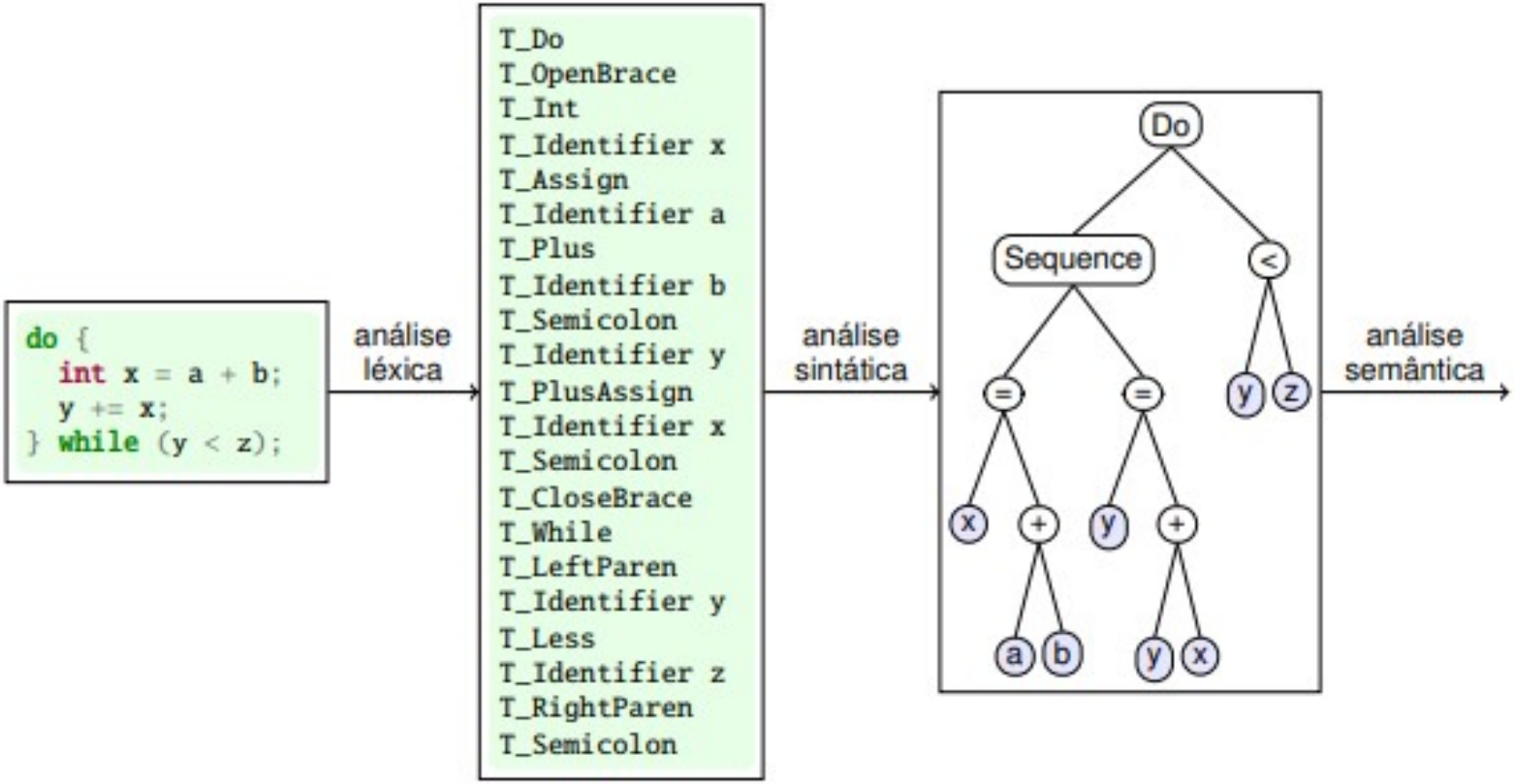
  - Interpretador
    - Não há criação de um programa em linguagem de máquina equivalente ao programa-fonte;



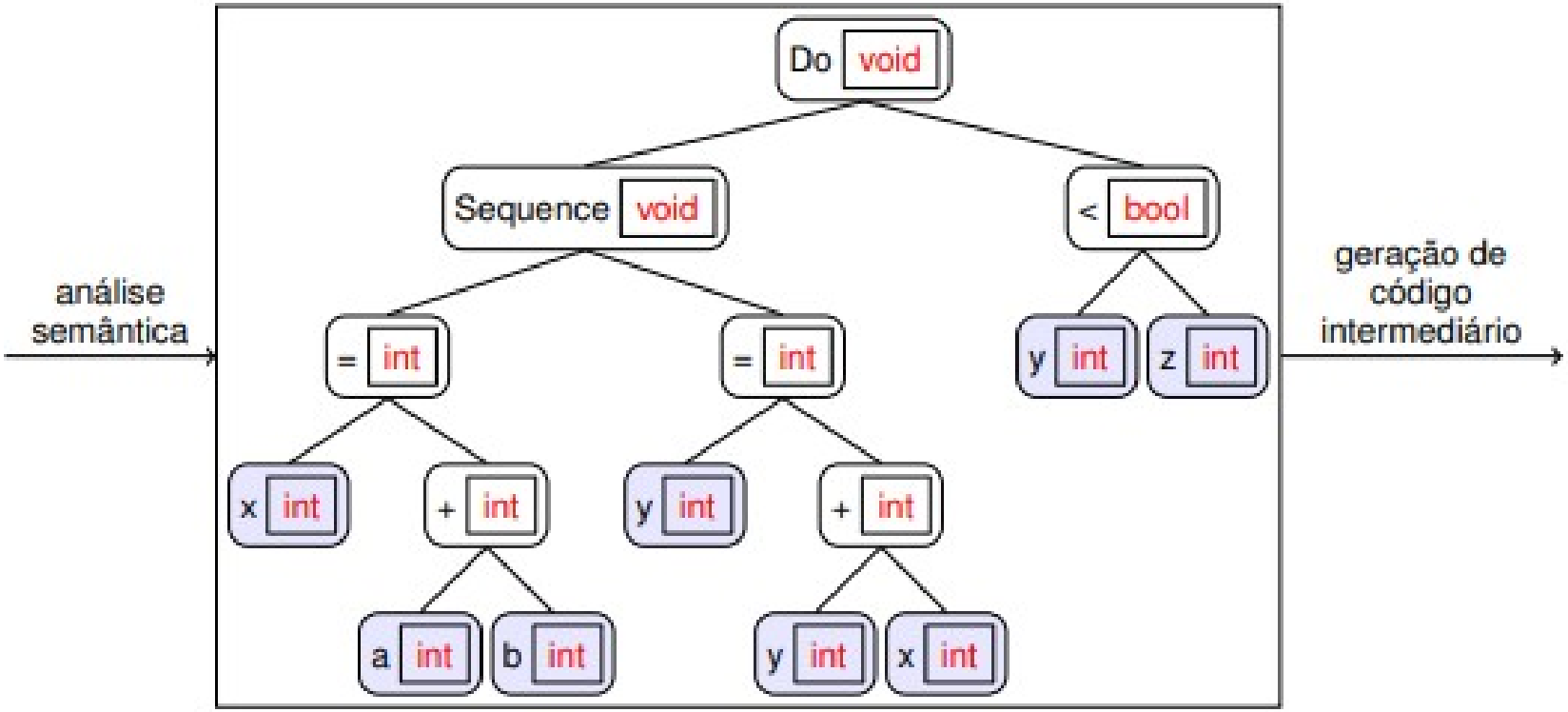
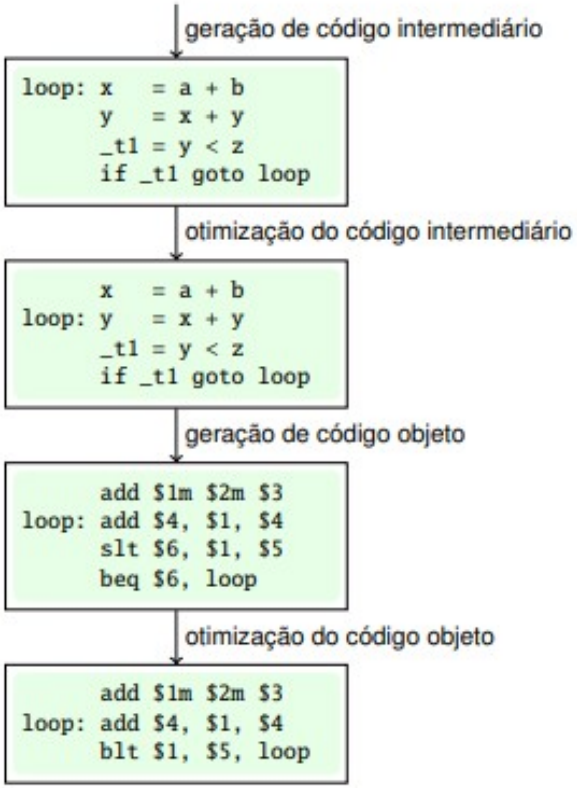
- **Processadores de linguagem: Compilação + interpretação**



- **Estrutura de um compilador moderno**



- **Estrutura de um compilador moderno**



- **Estrutura de um compilador moderno**